



WESTERN MICHIGAN
UNIVERSITY

College of Engineering and Applied Sciences

Course Name: Lab. Intro. To Microprocessors

Course Number: ECE 2510

Instructors:

[Christian Blanchard](#)

[Pravana Avuthu](#)

Homework #5

Student:

De Leon Santos, Sebastian Manuel – WIN: 577866168

Major: Computer Engineering

Task 1

```
2026\intro_microprocessors\lab\hm5\hm5.uvprojx - µVision [Non-Commercial Use License]
Debug Peripherals Tools SVCS Window Help

lt_1
task1.s
1 AREA myData, DATA, READWRITE
2 GPIO_B_BASE EQU 0x48000400
3 RCC_BASE EQU 0x40021000
4
5 GPIO_MODER equ 0x00
6 GPIO_IDR equ 0x10
7 GPIO_ODR equ 0x14
8
9 RCC_AHB2ENR equ 0x4C
10
11 AREA readData, DATA, READONLY
12 ALIGN
13 digit_select DCW 0x0E00 ; Thousands
14 DCW 0x0D00 ; Hundreds (CA2)
15 DCW 0x0B00 ; Tens (CA3)
16 DCW 0x0700 ; Ones (CA4)
17 ALIGN
18 ; E C E blank
19 word_ECE dcb 0x79, 0x39, 0x79, 0x00
20 ; 2 5 1 0
21 word_2510 dcb 0x5B, 0x6D, 0x06, 0x3F
22
23
24 AREA myCode, CODE, READONLY
25 EXPORT __main
26 ENTRY
27
28 enable_clock proc
29
30 ldr r2, =RCC_BASE
31 ldr r1, [r2, #RCC_AHB2ENR]
32 orr r1, r1, #0x06 ;0110 (bit 1 & bit 2)
33 str r1, [r2, #RCC_AHB2ENR]
34 bx lr
35
36 endp
37
38 setup_gpio proc
39
40 ; configuring port b 12 pins as output
41 ldr r0, =GPIO_B_BASE
42 ldr r1, [r0, #GPIO_MODER]
43 ldr r2, =0x00FFFFFF
44 bic r1, r1, r2
45 ldr r2, =0x00555555
46 orr r1, r1, r2
47 str r1, [r0, #GPIO_MODER]
48
49 bx lr
50 endp
51
52 __main proc
53
54 bl enable_clock
55 bl setup_gpio
56
57 loop
```

task1.s

```

55         bl      setup_gpio
56
57     loop
58
59     mux_lsec_ece    ldr      r9, =100
60
61         ldr      r0, =word_ECE
62         bl      display_word
63         subs    r9, r9, #1
64         bne     mux_lsec_ece
65
66         bl      display_blank
67         bl      delay_lsec
68
69         ldr      r9, =100
70
71     mux_lsec_2510   ldr      r0, =word_2510
72
73         ldr      r0, =word_2510
74         bl      display_word
75         subs    r9, r9, #1
76         bne     mux_lsec_2510
77
78         bl      display_blank
79         bl      delay_lsec
80
81         b       loop
82
83     endp
84
85     display_blank   proc
86         ldr      r0, =GPIO_B_BASE
87         ldr      r1, =0x00000FFF
88         ldr      r2, [r0, #GPIO_ODR]
89         bic     r2, r2, r1
90         ldr      r1, =0x00000F00
91         orr     r2, r2, r1
92         str     r2, [r0, #GPIO_ODR]
93         bx      lr
94     endp
95
96     display_word     proc
97         push    {r4-r8, lr}
98         mov     r4, r0                ; copying digit pointer to r4                ; r5 = loop index
99
100        ldr      r6, =GPIO_B_BASE
101        ldr      r7, =digit_select
102
103        mov     r5, #0                ; 0 through 3 loop
104    mux_loop        cmp     r5, #3
105                    bgt     mux_finish
106
107        ldrb     r1, [r4, r5]          ; load segment pattern in position
108                                    ; of digit to display
109
110        ldrrh    r2, [r7, r5, lsl #1] ; load the digit select half word
111
112        ; combination of segment pattern and digit selection

```

task1.s

```

101
102      mov     r5, #0                ; 0 through 3 loop
103 mux_loop    cmp     r5, #3
104             bgt     mux_finish
105
106             ldrb     r1, [r4, r5]    ; load segment pattern in position
107                                     ; of digit to display
108
109             ldrh     r2, [r7, r5, lsl #1] ; load the digit select half word
110
111             ; combination of segment pattern and digit selection
112             orr     r1, r1, r2
113             ldr     r3, =0x00000FFF
114             ldr     r0, [r6, #GPIO_ODR]
115             bic     r0, r0, r3
116             orr     r0, r0, r1
117             str     r0, [r6, #GPIO_ODR]
118
119             bl      delay_2ms
120
121             ; turn off display
122             ldr     r3, =0x00000FFF
123             ldr     r0, [r6, #GPIO_ODR]
124             bic     r0, r0, r3
125             ldr     r3, =0x00000F00    ; turn off digits    1000    0000 0000
126             orr     r0, r0, r3        ;                    4digits    8 segments
127             str     r0, [r6, #GPIO_ODR]
128
129             add     r5, r5, #1
130             b       mux_loop
131
132 mux_finish   pop     {r4-r8, pc}
133             endp
134

```

```

4
5 ;      delays
6
7 delay_2ms      PROC
8     push      {r4, r5}
9     mov       r4, #10
0     outer_2ms  mov       r5, #400
1     inner_2ms  subs      r5, r5, #1
2               bne       inner_2ms
3               subs      r4, r4, #1
4               bne       outer_2ms
5               pop       {r4, r5}
6               bx       lr
7               ENDP
8
9 delay_1sec      PROC
0     push      {r4, r5}
1     mov       r4, #500
2     outer_1sec  mov       r5, #2500
3     inner_1sec  subs      r5, r5, #1
4               bne       inner_1sec
5               subs      r4, r4, #1
6               bne       outer_1sec
7               pop       {r4, r5}
8               bx       lr
9               ENDP
0
1               ALIGN
2               END

```

Task 2

```
Welcome  C main.c  X
C: > Users > sebas > wmu_classes > spring2026 > intro_microprocessors > lab > hm5_task2 > C main.c > ...
1  #include <stdio.h>
2  // Y = x^3 - 24z^2 - 64m + 1024
3
4  int equation(int x, int z, int m) {
5      int y = (x * x * x) - (24 * z * z) - (64*m) + 1024;
6      return y;
7  }
8
9  void main() {
10     int x = 2, z = 5, m = 7;
11     int y = 0;
12     y = equation(x, z, m);
13
14     printf("Y = %d\n", y);
15 }
```

Task 3

a. Describe the problem in one paragraph.

In 1994 Intel released a new chip called the Intel Pentium. On this new processor, they wanted to improve the speed of floating-point division calculations, for this, they swapped the previous algorithm with the SRT algorithm which calculated 2 bits of the division in one cycle. This is better than the last algorithm which calculated only 1 bit per cycle. This algorithm uses a lookup table that has 1066 cells populated with -2, -1, 0, 1 or 2. But there was an error at the time of manufacturing, five values of +2 were not received by the lithography equipment making those spaces be filled with zeros. As a result, the calculations that used these five cells were wrong. Due to the SRT Algorithm, this small error escalates and can grant too many errors when working, which is not convenient for any problem solving in the real world.

b. Did Intel fix the problem?

Intel didn't dedicate itself to replacing already sold microprocessors. They told the customers that they can get a replacement if they report that the FDIV issue really affects them. (Pentium FDIV bug, s.f.)

c. How often can a user see the problem?

First, Intel believed an average user would last 27,000 years to find the bug. Later, the firm concluded that an average spreadsheet user could expect a glitch every 24 days (Washington Technology, 1994). For game developers like John Romero and Michael Abrash it was even more frequent found on their work and easily replicable by the latter on their game engine. So, depending on the use of the machine, it was more or less frequent.

d. How much did it cost Intel Co.?

Intel stated in their 1994 annual report that this error took a \$475 million pretax charge to cover replacement and write-off of the microprocessors. (Intel Corporation, 1994)

e. How did this engineering problem affect the corporate's reputation?

This damaged Intel's reputation, leading to widespread criticism for its handling of the bug and a notable decline in stock prices.

f. What is corporate responsibility in disclosing bugs in a product?

Corporate responsibility in disclosing bugs involves being transparent about product flaws to maintain trust and ensure users can make informed decisions about their purchases.

References

Intel Corporation. (1994). *1994 Annual Report*. Retrieved from <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2025-05/history-1994-annual-report.pdf>

Pentium FDIV bug. (n.d.). Retrieved from Wikipedia:
https://en.wikipedia.org/wiki/Pentium_FDIV_bug#cite_note-5

Washington Technology. (1994, December 22). Understanding the Pentium Disaster. *Washington Technology*.